

# Final individual assignment: Flow field through a porous media

Michaela Ivanova  
1685112  
Data Science  
m.ivanova@student.tue

## 1. Introduction

In this assignment we consider the laminar flow of a viscous fluid through a porous medium, like a “sponge”. The porous medium extends along the interior of a long channel (length  $L$ , [m]). The fluid moves under the action of a pressure drop ( $\Delta P$ , [Pa]). Our objective is to design and train a feed-forward neural network that accurately predicts the transversal flow rate field,  $q(x_i, y_j)$  [ $\text{ms}^{-1}$ ], through the given cross sections. Together with the cross section, the flow rate field is determined by parameters such as: pressure drop  $\Delta P$  over the channel ([Pa], the length  $L$  of the channel ([m] the fluid viscosity  $\mu$  ([Pa · s]), the area  $\Delta A = \Delta x \Delta y$  of individual pixels in the binary image ([ $\text{m}^2$ ]). The overall flow per second through the porous medium,  $Q$  [ $\text{m}^3\text{s}^{-1}$ ], relates to the flow field  $q$  as

$$Q = \sum_{ij} q(x_i, y_j) \Delta A. \quad (1)$$

## 2. Part A - Physics of the problem

In this section, we specify the task as a physics problem and as machine learning problem.

First, our training dataset consists of 1500 samples, which are composed of 32×64 binary grid representing a porous cross-section. Each pixel location is either open (pixel value = 1) if the fluid can flow through it, or closed (pixel value = 0) if it is impermeable to the flow. It has the shape [1500, 32, 64]. For each grid, we also have four physical parameters: pressure drop, length, viscosity, and pixel area. Moreover, we have labels, which show the true flow rate field  $q(x_i, y_j)$  for each 32×64 grid.

To characterize the dataset, an example of the input grid with the calculated output for  $Q$  by the given formula in Equation 1 is given in Figure 1.

Moreover, we computed the porosity for each sample and obtained the following descriptive statistics: Mean: 0.1836, Std: 0.2099, Min: 0.0034, Max: 0.8643. This gives us a better understanding of the distributions of the samples.

**2.0.1. Dimensionality and Scaling.** Since our task involves predicting fluid transport through porous cross-sections, we need to start from a fundamental physical description of



Figure 1: Example Input with Output  $Q$ : 32×64 binary grid

flow in porous media. The hydrodynamics of the fluid flow in the porous media can be modeled by Darcy’s law [2]. In the absence of gravitational forces, and assuming a homogeneously permeable medium, Darcy’s law describes a simple proportionality between the volumetric flow rate  $Q$ , and the pressure drop  $\Delta p$  across the medium. The flow rate depends on the permeability  $k$  of the medium, the dynamic viscosity  $\mu$  of the fluid, the length  $L$  over which the pressure drop is applied, and the total cross-sectional area  $A$  [3]. The classical formulation of Darcy’s law reads:

$$Q = \frac{kA}{\mu L} \Delta p \quad (2)$$

We are using this expression to gain physical intuition for the scaling of the flow. While in our dataset the permeability  $k$  is unknown and varies spatially, it is implicitly encoded by the structure of the binary grid.

That is why we would need adapt this formula in order to take into account the dependencies in our data, where the flow field  $q(x, y)$  is predicted per-pixel based on the grid structure and physical parameters. So now we will draw an analogy between Darcy’s law and Equation 1 provided for the flow rate.

Assuming the form of Darcy’s law holds locally, we extract a characteristic scaling for the pixel-level flow field  $q(x, y)$ :

$$q(x, y) \propto \frac{\Delta P \Delta A}{\mu L} \quad (3)$$

We can keep this in mind while we are providing reasoning for further dimensionality analysis.

Although the input grid cannot be included in the dimensional analysis, since it is a unitless binary image encoding structural information — it is still used for determining the

spatial variation of the flow. The geometric effect would be learned from the grid through a neural network.

The full model is then expressed as:

$$f(\text{geometry}) \times f(\Delta P, \mu, L, \Delta A) \approx q(x, y)$$

For the **homogeneous case**, we need to express the features and labels in the same fundamental physical quantity. We aim to express the flow field  $q(x, y)$  as a function of the input parameters, all sharing compatible physical dimensions. Specifically, we consider the function:

$$f(\Delta P, \mu, L, \Delta A) \approx q(x, y) \quad (4)$$

The units of each variable are:

$$f([\text{Pa}], [\text{Pa} \cdot \text{s}], [\text{m}], [\text{m}^2]) \approx [\text{m} \cdot \text{s}^{-1}]$$

One possible way to do this would be to combine these inputs to produce a function with the correct output dimension  $[\text{m}/\text{s}]$ . To this end, it suffices to use the resulting Equation 3 for our input:

$$f(\text{geometry}) \times f\left(\frac{\Delta P \cdot \Delta A}{\mu \cdot L}\right) \approx q(x, y) \quad (5)$$

Then if we substitute the units into the expression we get:

$$f(\text{geometry}) \times f\left(\frac{[\text{Pa}] \cdot [\text{m}^2]}{[\text{Pa} \cdot \text{s}] \cdot [\text{m}]} = \frac{[\text{m}]}{[\text{s}]}\right) \approx [\text{m}/\text{s}]$$

This formulation is homogeneous of degree 1.

For the **dimensionless formulation**, we need to get rid of all physical units. That can be done by scaling the homogeneous formulation. The scaled output flow  $\tilde{q}(x, y)$  is defined as:

$$\tilde{q}(x, y) = \frac{q(x, y)}{\frac{\Delta P \cdot \Delta A}{\mu \cdot L}}$$

Rewriting:

$$\tilde{q}(x, y) = q(x, y) \cdot \frac{(\mu \cdot L)}{(\Delta P \cdot \Delta A)} \quad (6)$$

Now, we can substitute the units:

$$\tilde{q}(x, y) = [\text{m}/\text{s}] \cdot \frac{[\text{Pa} \cdot \text{s}] \cdot [\text{m}]}{[\text{m}^2 \cdot \text{Pa}]} = \frac{[\text{m}]}{[\text{s}]} \cdot \frac{[\text{s}]}{[\text{m}]} = 1$$

Thus, the output  $\tilde{q}(x, y)$  is dimensionless.

Now we will also scale the input from Equation 5 in the same way, which would ultimately give:

$$f\left(\frac{\Delta P \cdot \Delta A}{\mu \cdot L} \cdot \frac{\mu \cdot L}{\Delta P \cdot \Delta A}\right) = f(1)$$

So, to put everything together for the dimensionless formulation we get:

$$f(\text{geometry}) \approx \tilde{q}(x, y)$$

This would mean that the system would only get the geometry of the binary grid with no parameters for the input (only bias), and a scaled output.

**2.0.2. Symmetries.** Now we can explain the symmetries we consider. A model is said to be invariant to a transformation if the output remains unchanged even when the input is transformed, by e.g. shifting or scaling. An equivariant model would be such that the output transforms in the same way as the input. In other words, when a transformation is applied to the input, the output transforms in a predictable and consistent way [5].

Considering the physics of the problem, the 32x64 binary grid would have the following symmetries:

- **Rotation Equivariance:** If we rotate the grid, the predicted flow field should also rotate by the same amount.
- **Flip Equivariance:** If we flip the grid horizontally or vertically, the flow field should also reflect the same transformation.

Other symmetries (e.g. random permutations of the pixels) are not relevant. Fluid flow relies on spatial dependencies in the grid, for example, if there are many neighboring cells that are open. Randomly permuting pixels would remove these spatial dependencies, making it impossible to model the flow in the medium.

Note! Since the grid is a rectangle, Rotation Equivariance could change the physics of the problem. For instance, if we rotate the grid by 90 degrees, then we are swapping the width and height. This would result in a different physical configuration since what was originally a wider grid (in the x-direction) is now taller (in the y-direction). As shown by [7], misalignment between the flow direction and the permeability axis (i.e., rotating the permeability tensor) leads to significant changes in both the fluid velocity and stress distribution, supporting that grid rotation can alter the underlying physics of flow through anisotropic porous media. (Here we are making an assumption that the porous medium could be anisotropic, due to the random structure of the binary grids, so many samples likely exhibit anisotropic behavior).

Ultimately, we can use Flipping (horizontal and vertical) and a combination of Flipping + Rotation of 180 degrees. Essentially, Flipping + Rotation of 180 degrees corresponds to flipping along the x-axis + flipping along the y axis.

Moreover, for the 32x64 binary grid there are no symmetries that are going to be invariant, since the predictions are per pixel and the output would always need to transform in the same way with the grid.

Furthermore, most of the existing neural networks cannot maintain transformation invariance when handling geometric data [6].

If we consider the input parameters, there are different ways to scale them. In the **homogeneous formulation** the physical parameter  $\frac{\Delta P \cdot \Delta A}{\mu \cdot L}$  from Equation 5 can be scaled without changing the relationship between the input and the output. So, for example if we want to multiply the viscosity  $\mu$  by a scalar  $\lambda$ , the entire formula would be scaled by  $\frac{1}{\lambda}$ . The formula then becomes  $\frac{\Delta P \cdot \Delta A}{(\mu \cdot \lambda) \cdot L} = \frac{1}{\lambda} \cdot \frac{\Delta P \cdot \Delta A}{\mu \cdot L}$ . Since the output  $q(x, y)$  would not change, this means that the homogeneous formulation is scaling invariant.

If we apply the same logic for the **dimensionless formulation**,

if we scale the viscosity  $\mu$  by a scalar  $\lambda$ , the output  $\tilde{q}(x, y)$  from Equation 6 will change accordingly. Specifically, the scaling factor will impact the flow field, and the new dimensionless output becomes:  $\tilde{q}(x, y) \rightarrow \frac{1}{\lambda} \cdot \tilde{q}(x, y)$ . So, the dimensionless formulation is scaling equivariant.

**2.0.3. Loss functions.** The **Mean Squared Error (MSE)** loss function is commonly used for regression tasks. It is defined as the average of the squared differences between the predicted values  $\tilde{q}_{\text{pred}}(x_i, y_i)$  and the true values  $\tilde{q}_{\text{true}}(x_i, y_i)$  [4]:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (\tilde{q}_{\text{pred}}(x_i, y_i) - \tilde{q}_{\text{true}}(x_i, y_i))^2$$

where  $N$  is the total number of pixels,  $\tilde{q}_{\text{pred}}(x_i, y_i)$  is the predicted value, and  $\tilde{q}_{\text{true}}(x_i, y_i)$  is the true value. MSE squares the errors, which means larger errors are penalized more heavily, making it sensitive to outliers.

The **Mean Absolute Error (MAE)** loss function calculates the average of the absolute differences between the predicted and true values [4]. It is defined as:

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |\tilde{q}_{\text{pred}}(x_i, y_i) - \tilde{q}_{\text{true}}(x_i, y_i)|$$

where  $N$  is the total number of pixels,  $\tilde{q}_{\text{pred}}(x_i, y_i)$  is the predicted value, and  $\tilde{q}_{\text{true}}(x_i, y_i)$  is the true value.

MAE is less sensitive to outliers compared to MSE, as it does not square the differences and treats all errors equally.

Since we also want the model to respect symmetries in the input, we introduce an additional **symmetry-aware loss term**. For example, a prediction for a horizontally flipped input should match the horizontally flipped prediction of the original input.

Let:

- $q_{\text{orig}} = f(G)$ : the model prediction on the original input grid  $G$ ,
- $q_{\text{flip}} = f(G_{\text{flip}})$ : the prediction on the horizontally flipped input  $G_{\text{flip}}$ ,
- $\text{flip}(q_{\text{orig}})$ : the horizontally flipped version of the original prediction.

Then, the symmetry loss is defined as:

$$\text{SymLoss} = \|q_{\text{flip}} - \text{flip}(q_{\text{orig}})\|^2$$

This term penalizes differences between the prediction on the flipped input and the flipped prediction on the original input. The total loss used during training combines the standard prediction loss with the symmetry loss:

$$\text{TotalLoss} = \text{pred\_loss} + \lambda \cdot \text{sym\_loss}$$

where  $\lambda$  is a weighting factor that controls how strongly the symmetry constraint is enforced.

Last but not least, the loss that is used in our implementations (and in the Kaggle competition) is the Relative Error Loss (RE), defined as

$$E = \frac{1}{|\text{dataset}|} \sum_{i \in \text{dataset}} \frac{1}{N_X N_Y} \sum_{x, y} \left| \frac{q_i^T(x, y) - q_i(x, y)}{q_i^T(x, y) + \epsilon} \right|, \quad (7)$$

where  $q_i(x, y)$  is the predicted flow rate value of the  $i$ -th datapoint at pixel location  $(x, y)$ ,  $q_i^T(x, y)$  is the corresponding ground truth value, and  $\epsilon$  is a small constant to avoid division by zero.

### 3. B - Network design

In order for the network to learn the symmetries, we can either do Data Augmentation, Disambiguate the symmetry, or Hardwire the symmetry.

**3.0.1. Data Augmentation.** In CNNs, translation equivariance is inherently achieved through the convolutional operation. In fact, if the input image is translated to the right by a certain amount, the feature maps generated by convolutional layers are shifted by the same amount and direction [8].

However, CNNs are not naturally equivariant to other transformations relevant to our problem, such as rotations and reflections. That is why we can use data augmentation to expand the training dataset with transformed versions of the original images [9]. For our porous media flow prediction task, this would involve data augmentations such as Rotation and Flipping as stated in Section 2.0.2 (without 90 degree rotations). However, data augmentations enlarge the dataset, which would make the training of the models more computationally expensive.

**3.0.2. Disambiguation.** Due to the massive number of possible transformations, many times data augmentation cannot guarantee effectiveness and works poorly in practice [6]. This approach introduces a lot of variance in the network and might make it hard for it to learn the symmetries.

On the other hand, disambiguating the symmetry approach involves enforcing inherent symmetries in the data by casting all instances of the problem into a (pre)defined format. One possible way to do this would be to choose a specific configuration of the images. Although the images have different structure, we can try applying a simple rule, e.g. use the image orientation that has the maximum number of open pixels in the bottom left quadrant (here, bottom left is randomly chosen, any other quadrant is also possible).

**3.0.3. Different designs of feedforward neural networks.** **Convolutional Neural Networks** are a natural choice for modeling flow through porous media, due to their ability to learn local spatial patterns [15]. CNNs employ local receptive fields to focus on specific regions of the input data, enabling them to capture local patterns and spatial relationships. They also utilize hierarchical processing, with

multiple layers that can extract increasingly complex features [16]. Moreover, they are known as shift invariant, because of the the shared-weight architecture of the convolution kernels or filters that slide along input features and provide translation-equivariant responses (the feature maps) [17]. The weight sharing also reduces the number of trainable network parameters and helps the network with generalization and prevents overfitting [18]. On the other hand, CNNs have some limitations. For example, they prioritize local patterns over global structures, which makes them less effective when objects have complex spatial dependencies. Moreover, they rely on pooling layers and strided convolutions to achieve translation invariance, which helps recognize objects regardless of their position, but this process might discard precise spatial information. Last but not least, they can be very computationally expensive, especially the deeper architectures [19].

Using the **Visual Geometry Group** is another approach we can take. It has a classical convolutional neural network architecture and it was developed to increase the depth of CNNs to increase the model performance [20].

The main difference is that it replaces the large kernel-sized filters with several very small  $3 \times 3$  kernel-sized filters one after the other [21]. In the VGG architecture, there are also  $1 \times 1$  convolution filters acting as a linear transformation of the input, followed by a ReLU unit. The convolution stride is fixed at 1 pixel to keep the spatial resolution preserved after convolution [20]. Spatial pooling is carried out by five max-pooling layers, which follow some of the conv. layers. Max-pooling is performed over a  $2 \times 2$  pixel window, with stride 2. Spatial pooling in the VGG architecture is implemented using five max-pooling layers, each applied after specific convolutional blocks. These pooling operations use a  $2 \times 2$  window with a stride of 2. All the hidden layers use ReLU. A stack of convolutional layers is followed by three Fully-Connected (FC) layers with a different number of channels. The final layer is the soft-max layer [21]. In general, the architecture's simplicity is what makes this network more appealing and it is straightforward to implement. However, because of its depth and number of fully connected layers, the VGG16 model is a huge network, which makes it very computationally heavy and time-consuming [20]. Moreover, its 138 million parameters lead to exploding gradients problem [22].

Another model that we can consider using is **U-Net**. It is one of the most widely used neural networks for image segmentation and it is a Fully Convolutional Neural Network Model [24]. The basic structure of a U-net architecture consists of two paths. The first path is the contracting path (known as encoder), which is similar to a regular convolution network and provides classification information. The second is an expansion path (known as decoder), consisting of up-convolutions and concatenations with features from the contracting path [25].

The main canonical task performed by most CNNs is to classify the whole image into a single label. However, classification networks fail to provide pixel-level context information [25], which is necessary in the case of predicting

the flow in porous medium.

One key modification in the U-Net architecture is the use of a large number of feature channels in the upsampling (expansive) path. This design enables the network to effectively propagate contextual information to higher-resolution layers during reconstruction. As a result, the expansive path becomes largely symmetric to the contracting (encoder) path, giving the network its characteristic U-shaped structure. Notably, the architecture operates fully convolutionally, using only the valid part of each convolution and omitting fully connected layers entirely. [26]. Moreover, U-Net utilizes skip connections. Each skip connection unit receives all of the feature maps from all previous units at the same level as well as an upsampled feature map from its immediate lower unit. This way the loss of semantic information between the two paths is minimized [25]. Last but not least, U-Net is effective even with a limited data set [24], which is the case in with our dataset since we only have 1500 samples. That is why, U-Net is expected to have the best performance out of the 3 options.

Now that we have considered all the options of CNN/VGG/U-Net we can introduce the concept of *Scaling of the receptive fields*. The receptive field is defined as the size of the region in the input that produces the feature [23]. The receptive field size can be computed with the recursive equation:

$$r_{l-1} = s_l \cdot r_l + (k_l - s_l) \quad (8)$$

where  $r_l$  is the receptive field at layer  $l$ ,  $s_l$  is the stride at layer  $l$ ,  $k_l$  is the kernel size at layer  $l$ .

And then in order to solve the recursive equation for  $k_l$  and  $s_l$  we get:

$$r_0 = \sum_{l=1}^L \left( (k_l - 1) \cdot \prod_{i=1}^{l-1} s_i \right) + 1 \quad (9)$$

This can help us track how much of the input image contributes to each output unit.

**3.0.4. Hard wiring the symmetry.** As previously stated, CNNs are inherently equivariant to translations, but they are not naturally equivariant to rotations or reflections. In order to **hardwire the symmetries in a CNN**, we can consider a Group Equivariant CNN (G-CNN). G-CNNs generalize convolution to be equivariant under any finite group  $G$  [10]. The set of symmetry transformations considered in our problem forms a finite group under composition, consisting of horizontal flip, vertical flip, and a  $180^\circ$  rotation (equivalent to applying both flips). This group is isomorphic to the Klein four-group, commonly denoted  $V_4$ , which is abelian [11].

In the context of (G-CNNs), we define a function  $f : G \rightarrow R^n$  to be equivariant to a group  $G$  if:

$$f(T_g x) = T'_g f(x), \quad \forall g \in G$$

where  $T_g$  is a transformation acting on the input and  $T'_g$  is the corresponding transformation acting on the output.

In a G-CNN, feature maps are modeled as functions on the group  $G$ . The group convolution (also called group correlation) at the first layer is defined as:

$$[f \star \psi](g) = \sum_{y \in Z^2} \sum_k f_k(y) \psi_k(g^{-1}y) \quad (10)$$

In deeper layers, the inputs and filters are also functions on the group  $G$ , and the group convolution becomes:

$$[f \star \psi](g) = \sum_{h \in G} \sum_k f_k(h) \psi_k(g^{-1}h) \quad (11)$$

We need to note that although the equivariance condition is expressed using the same formula for both the first-layer and deeper-layer group convolutions

$$[L_u f] \star \psi = L_u [f \star \psi]$$

where  $L_u f$  is the function  $f$  that has been transformed by group element  $u$ .

However, the interpretation of the transformation operator  $L_u$  differs. In the first layer, the input  $f$  is defined on the image domain  $Z^2$ , so  $L_u f$  corresponds to spatial transformations such as flips or rotations of the input image. In contrast, in deeper layers, the feature maps are functions on the group  $G$  itself, and  $L_u$  acts by permuting the group elements accordingly. This formulation makes sure that equivariance is preserved through all layers of the network. As a result, the model can naturally generalize over symmetric inputs without requiring data augmentation or learning from all symmetric variants [10].

In order to **hardwire the symmetries in a U-net** we can use the insights from the Group Equivariant CNNs [10] and apply other group operations, which preserve equivariance to such transformations throughout the network. So, starting with group equivariant convolution, we are going to use the same logic from Equations 10 and 11, using our group  $G$  which we defined to be the Klein four group  $V_4$ . Then, in the GU-Net architecture, for the upsampling arm, before each layer, the feature map is first upsampled from the layer below by 2 using nearest-neighbor interpolation, which preserves equivariance [13]. Max pooling is then applied with stride 2 independently to each group channel (in our case  $V_4$ ), which preserves the equivariance property, as the pooling operation commutes with the group action  $PL_g = L_g P$ . Residual blocks are also equivariant [10]. In the end, we need to apply a projection layer [13] to transform the orientation channels to the two-dimensional grid of the output mask [14] with a  $(G \rightarrow Z^2)$ -convolution. In order to do that, we can average the feature map for each filter over our group [13].

**3.0.5. Types of output.** Based on the problem physics, it is possible that sometimes the network has non-physical outputs, for example, outputting unrealistic flow, or flow that was not there at the first place. So, for instance, if the input has a closed pixel and the network predicts an open pixel with some flow in it, this is impossible. One thing we can do in order to prevent that, would be to apply a binary mask.

It would mask the predicted flow field so that flow is only preserved in the permeable regions. We would multiply the predicted flow by the binary image.

## 4. C - Network Implementation

### Preprocessing

To implement the **homogeneous formulation**, we define a custom PyTorch Dataset class named `PorousFlowHomogeneousDataset`. This class takes the binary geometry grids, the physical parameters, and the target flow fields as input. We first make a scalar as we have previously explained:

```
1 self.scalars = (params_df['delta_p'] * params_df['
2                 delta_A']) /
3                 (params_df['visc'] * params_df['L'
4                 ])
5 scaled_grid = grid * scalar
```

And then the scalar is multiplied with the binary grid. Since the grid consists only of 0s and 1s only the 1s are affected. The scaled grid is converted into a tensor and reshaped.

```
1 grid_tensor = torch.tensor(scaled_grid).unsqueeze
2                 (0)
3 label = torch.tensor(self.flow_fields[idx])
```

The flow fields are also returned as labels.

To implement the **dimensionless formulation** we can reuse some the code for the homogeneous formulation and just make small changes. First we compute inverse of the homogeneous formulation scaling factor, and then the target output is scaled:

```
1 self.scales = (mu * L) / (delta_p * delta_A)
2 self.flow_fields = flow_fields.astype(np.float32)
3                 * self.scales
```

### Symmetries

For the **data augmentation** we are using the function `apply_symmetry_augmentations`. It generates flipped versions (horizontal, vertical and combined) of both the input geometry and the label.

```
1 g, l = grid, label
2 if flip == 'h':
3     g = np.flip(g, axis=1)
4     l = np.flip(l, axis=1)
5 elif flip == 'v':
6     g = np.flip(g, axis=0)
7     l = np.flip(l, axis=0)
8 elif flip == 'hv':
9     g = np.flip(np.flip(g, axis=0), axis
10                 =1)
11     l = np.flip(np.flip(l, axis=0), axis
12                 =1)
```

For disambiguating the symmetry we are using a function called `select_best_symmetry_configuration`, which is an extension of the data augmentation function:

```

1 # Evaluate bottom-left quadrant
2 h, w = g.shape
3 bl_quadrant = g[h//2:, :w//2]
4 score = np.sum(bl_quadrant == 1)
5
6 if score > best_score:
7     best_score = score
8     g_scaled = g * scalar
9     best_config = (g_scaled.copy(), 1.copy
10 ())
11
12 return best_config

```

It returns the "best" configuration of the image, based on the bottom-left quadrant rule that we defined previously.

## Models

All the models' architectures can be found in Appendix B.

The architecture of the CNN is simple, composed of 5 convolutional layers with ReLU activations. The VGG-like architecture is also more simple than the usual VGG16 architecture, as it has 7 convolutional layers, no max pooling and no fully connected layers.

The architecture of the U-Net model is once again simplified - there are 2 downsampling blocks, bottleneck and 2 upsampling, and the number of channels reaches a maximum of 32.

For **hardwiring the symmetry** we are using the U-Net architecture that can be seen in Appendix C. It has downsampling, a bottleneck, and upsampling stages, with skip connections linking corresponding layers in the encoder and decoder. Then we define a

ApplySymmetry Module to apply the four symmetry transformations to the input. After that we use the GroupAverage Module which aggregates the outputs from all symmetry-transformed inputs by averaging them (group pooling). In the ImageMask we mask the predicted flow so that it only predicts flow in the permeable regions. Lastly, the UNet\_GA model is designed to process all symmetric versions of an input image and average the predictions to produce a symmetry-consistent output. It would act like a "wrapper" on the other U-Net. The total trainable parameters in this model are 29,321.

## Model Performances

The general model performances can be seen in Appendix A. The performances in terms of the different configurations of the types of dataset and dimensional scaling can be seen from Tables 1, 3, 2. Moreover, the RE loss function can be seen over each epoch in Figures 3, 2, 4. It should be noted that the training of the models has been done over different number of epochs, since we have been using EarlyStopping with a patience of 20. Moreover, lr\_scheduler.ReduceLROnPlateau has been used to reduce the learning rate during training when our metric has stopped improving. It can be seen from the tables, that for all the models, consistently the configuration with the

homogeneous formulation and data augmentation achieved the best loss. Furthermore, the standard U-Net has the best performance in general. The U-Net with Group Averaging has been used for the Kaggle Competition and its performance can be seen in Figure 5. It has reached a Validation loss of 0.079 and a Kaggle test loss of 0.053. There are mostly no signs of overfitting in all of the models.

## 5. Conclusion

We explored CNN, VGG, and U-Net models for predicting flow through porous media, with the homogeneous formulation and data augmentation yielding the best results overall. U-Net, particularly with Group Averaging, performed the strongest, achieving a validation loss of 0.07.

However, across all models, the loss remains above the desired target, which means there is room for improvement. Future work could focus on bigger and more complex architectures, better regularization constraints to improve accuracy and generalization.

## References

- [1] H. Kopka and P. W. Daly, *A Guide to L<sup>A</sup>T<sub>E</sub>X*, 3rd ed. Harlow, England: Addison-Wesley, 1999.
- [2] M. Rafiqul Islam, "Overview of Reservoir Simulation of Unconventional Reservoirs," in *Unconventional Gas Reservoirs*, Elsevier, 2015, pp. 487–547.
- [3] Wikipedia contributors, "Darcy's law," *Wikipedia, The Free Encyclopedia*, [https://en.wikipedia.org/wiki/Darcy%27s\\_law](https://en.wikipedia.org/wiki/Darcy%27s_law).
- [4] Adithya Prasad Pandelu, *Day 47: Loss Functions — Cross-Entropy, MSE, and MAE*, Medium, 2024, <https://medium.com/@bhatadithya54764118/day-47-loss-functions-cross-entropy-mse-and-mae-bdacebc7b428>.
- [5] W. Pai, "Invariance v.s equivariance in computer vision," *Medium*, Oct. 2021. [Online]. Available: <https://medium.com/@weichenpai/invariance-v-s-equivariance-in-computer-vision-ed6663ee9a35>
- [6] Z. Zhang, X. Wang, Z. Zhang, P. Cui, and W. Zhu, *Revisiting Transformation Invariant Geometric Deep Learning: Are Initial Representations All You Need?*, arXiv preprint arXiv:2112.12345, 2021. [Online]. <https://arxiv.org/abs/2112.12345>.
- [7] V. K. Verma and A. F. Ansari, "Darcy-Brinkman Flow of a Micropolar Fluid through an Anisotropic Porous Channel," *Journal of Computational and Theoretical Transport*, vol. 53, no. 7, pp. 575–594, Oct. 2024, doi: 10.1080/23324309.2024.2413187.
- [8] Baeldung. *Translation Invariance and Equivariance in Convolutional Neural Networks*. Available at: <https://www.baeldung.com/cs/translation-invariance-equivariance>.
- [9] Shorten, C. and Khoshgoftaar, T. M., "A survey on Image Data Augmentation for Deep Learning", *Journal of Big Data*, vol. 6, no. 1, pp. 1–48, 2019. <https://doi.org/10.1186/s40537-019-0197-0>
- [10] Cohen, T. S., & Welling, M. (2016). Group equivariant convolutional networks. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)* (Vol. 48, pp. 2990–2999). <https://arxiv.org/abs/1602.07576>
- [11] Wikipedia contributors. Klein four-group. *Wikipedia, The Free Encyclopedia*. Available at: [https://en.wikipedia.org/wiki/Klein\\_four-group](https://en.wikipedia.org/wiki/Klein_four-group).
- [12] Sangalli, M., Blusseau, S., Velasco-Forero, S., & Angulo, J. (2022). Scale-Equivariant U-Net. In *Proceedings of the British Machine Vision Conference (BMVC)*. <https://bmvc2022.mpi-inf.mpg.de/0763.pdf>

- [13] Chidester, B., Ton, T., Tran, M., Ma, J., Do, M. (2019). Enhanced Rotation-Equivariant U-Net for Nuclear Segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. [https://openaccess.thecvf.com/content\\_CVPRW\\_2019/papers/CVMI/Chidester\\_Enhanced\\_Rotation-Equivariant\\_U-Net\\_for\\_Nuclear\\_Segmentation\\_CVPRW\\_2019\\_paper.pdf](https://openaccess.thecvf.com/content_CVPRW_2019/papers/CVMI/Chidester_Enhanced_Rotation-Equivariant_U-Net_for_Nuclear_Segmentation_CVPRW_2019_paper.pdf)
- [14] Linmans, J., Winkens, J., Veeling, B. S., Cohen, T. S., & Welling, M. (2018). Sample Efficient Semantic Segmentation using Rotation Equivariant Convolutional Networks. *ICML Workshop: Towards learning with limited labels: Equivariance, Invariance, and Beyond*. Available at: <https://arxiv.org/abs/1807.00583>
- [15] DataDatum. (2019). Deep Learning in 5 Minutes (Part 1): Convolutional Neural Networks. [https://medium.com/@data\\_datum/deep-learning-in-5-minutes-part-1-convolutional-neural-networks-13ca4e0b486f](https://medium.com/@data_datum/deep-learning-in-5-minutes-part-1-convolutional-neural-networks-13ca4e0b486f)
- [16] Maurya, A. (2023). CNN Unveiled: Exploring Convolutional Neural Networks. <https://medium.com/@ahaansinghmaurya/cnn-unveiled-exploring-convolutional-neural-networks-3a202524b701>
- [17] Wikipedia contributors. Convolutional neural network. *Wikipedia, The Free Encyclopedia*. [https://en.wikipedia.org/wiki/Convolutional\\_neural\\_network](https://en.wikipedia.org/wiki/Convolutional_neural_network).
- [18] Alzubaidi, L., Zhang, J., Humaidi, A. J., et al. (2021). Review of deep learning: concepts, CNN architectures, challenges, applications, future directions. *Journal of Big Data*, 8, 53. <https://doi.org/10.1186/s40537-021-00444-8>
- [19] Milvus. (2025). What are the limitations of CNN in computer vision? *Milvus AI Quick Reference*. Retrieved April 12, 2025, from <https://milvus.io/ai-quick-reference/what-are-the-limitations-of-cnn-in-computer-vision>
- [20] Viso.ai. (2023). VGG – Very Deep Convolutional Networks. *Viso Suite*. <https://viso.ai/deep-learning/vgg-very-deep-convolutional-networks/>
- [21] Simonyan, K., & Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. In *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/1409.1556>
- [22] Cseh, A. (2023). VGG (Visual Geometry Group) – Everything you need to know. *Medium*. <https://medium.com/@2100031234cseh/vgg-visual-geometry-group-everything-you-need-to-know-8931a7f0f7cf>
- [23] Araujo, A., Norris, W., & Sim, J. (2019). Computing Receptive Fields of Convolutional Neural Networks. *Distill*. <https://distill.pub/2019/computing-receptive-fields/>. 10.23915/distill.00021
- [24] DataScientest. (2023). *U-Net: Computer Vision's Neural Network*. Retrieved from <https://datascientest.com/en/u-net-computer-visions-neural-network>
- [25] Siddique, N., Sidike, P., Elkin, C., & Devabhaktuni, V. (2020). *U-Net and its variants for medical image segmentation: theory and applications*. arXiv preprint arXiv:2011.01118.
- [26] Wikipedia contributors. (2023). *U-Net*. Wikipedia. Retrieved from <https://en.wikipedia.org/wiki/U-Net>

## Appendix A. Model Performances

TABLE 1: Final RE Validation Loss for CNN, using Clean, Augmented and Disambiguated dataset

| Formulation   | Clean | Augmented   | Disambiguated |
|---------------|-------|-------------|---------------|
| Homogeneous   | 5.11  | <b>0.69</b> | 2.64          |
| Dimensionless | 38.76 | 0.72        | 15.44         |

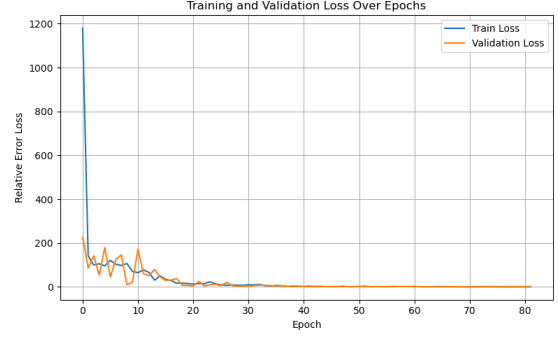


Figure 2: Train and Validation RE Loss for Homogeneous CNN with Augmentations



Figure 3: Train and Validation RE Loss for Homogeneous VGG with Augmentations



Figure 4: Train and Validation RE Loss for Homogeneous U-Net with Augmentations

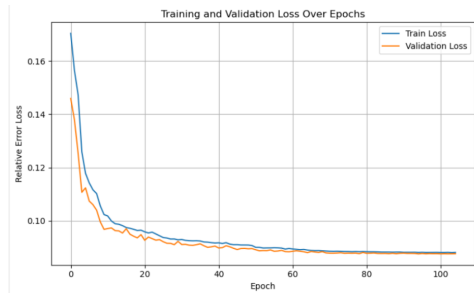


Figure 5: Train and Validation RE Loss for Homogeneous U-Net with Group Averaging

TABLE 2: Final RE Validation Loss for VGG, using Clean, Augmented and Disambiguated dataset

| Formulation   | Clean | Augmented   | Disambiguated |
|---------------|-------|-------------|---------------|
| Homogeneous   | 1.92  | <b>1.60</b> | 2.31          |
| Dimensionless | 4.55  | 5.97        | 7.19          |

TABLE 3: Final RE Validation Loss for U-Net, using Clean, Augmented and Disambiguated dataset

| Formulation   | Clean | Augmented   | Disambiguated |
|---------------|-------|-------------|---------------|
| Homogeneous   | 0.53  | <b>0.33</b> | 0.67          |
| Dimensionless | 9.14  | 0.82        | 7.52          |

## Appendix B. Models

### CNN

```

1 self.cnn = nn.Sequential(
2     nn.Conv2d(1, 16, kernel_size=3,
3       padding=1),
4     nn.ReLU(),
5     nn.Conv2d(16, 32, kernel_size=3,
6       padding=1),
7     nn.ReLU(),
8     nn.Conv2d(32, 64, kernel_size=3,
9       padding=1),
10    nn.ReLU(),
11    nn.Conv2d(64, 32, kernel_size=3,
12      padding=1),
13    nn.ReLU(),
14    nn.Conv2d(32, 1, kernel_size=1)
15 )

```

### VGG

```

1 self.features = nn.Sequential(
2     nn.Conv2d(1, 32, kernel_size=3,
3       padding=1), nn.ReLU(),
4     nn.Conv2d(32, 32, kernel_size=3,
5       padding=1), nn.ReLU(),
6     nn.Conv2d(32, 64, kernel_size=3,
7       padding=1), nn.ReLU(),
8     nn.Conv2d(64, 64, kernel_size=3,
9       padding=1), nn.ReLU(),
10    nn.Conv2d(64, 128, kernel_size=3,
11      padding=1), nn.ReLU(),
12    nn.Conv2d(128, 128, kernel_size=3,
13      padding=1), nn.ReLU(),
14    nn.Conv2d(128, 64, kernel_size=3,
15      padding=1), nn.ReLU(),
16    nn.Conv2d(64, 1, kernel_size=1)
17 )

```

### U-Net

```

1 self.enc1 = nn.Sequential(
2     nn.Conv2d(1, 8, kernel_size=3, padding
3       =1), nn.ReLU(),
4     nn.Conv2d(8, 8, kernel_size=3, padding
5       =1), nn.ReLU()
6 )
7 self.pool1 = nn.MaxPool2d(2) # 32x64 to
8 16x32
9 self.dec2 = nn.Sequential(

```

```

10     nn.Conv2d(8, 16, kernel_size=3,
11       padding=1), nn.ReLU(),
12     nn.Conv2d(16, 16, kernel_size=3,
13       padding=1), nn.ReLU()
14 )
15 self.pool2 = nn.MaxPool2d(2) # 16x32 to 8
16 8x16
17 self.bottleneck = nn.Sequential(
18     nn.Conv2d(16, 32, kernel_size=3,
19       padding=1), nn.ReLU(),
20     nn.Conv2d(32, 32, kernel_size=3,
21       padding=1), nn.ReLU()
22 )
23 self.up2 = nn.ConvTranspose2d(32, 16,
24   kernel_size=2, stride=2)
25 self.dec2 = nn.Sequential(
26     nn.Conv2d(32, 16, kernel_size=3,
27       padding=1), nn.ReLU(),
28     nn.Conv2d(16, 16, kernel_size=3,
29       padding=1), nn.ReLU()
30 )
31 self.up1 = nn.ConvTranspose2d(16, 8,
32   kernel_size=2, stride=2)
33 self.dec1 = nn.Sequential(
34     nn.Conv2d(16, 8, kernel_size=3,
35       padding=1), nn.ReLU(),
36     nn.Conv2d(8, 8, kernel_size=3, padding
37       =1), nn.ReLU()
38 )
39 self.final = nn.Conv2d(8, 1, kernel_size
40   =1)
41 def forward(self, x):
42     x1 = self.enc1(x)
43     x2 = self.enc2(self.pool1(x1))
44     x3 = self.bottleneck(self.pool2(x2))
45     x = self.up2(x3)
46     x = self.dec2(torch.cat([x, x2], dim=1))
47     x = self.up1(x)
48     x = self.dec1(torch.cat([x, x1], dim=1))
49     return self.final(x).squeeze(1)

```

## Appendix C. Best Kaggle Model: GA Unet

Model Architecture for U-net for hardwiring the symmetry

```

1 class TinyUNetHomogeneous(nn.Module):
2     def __init__(self):
3         super(TinyUNetHomogeneous, self).__init__()
4         self.enc1 = nn.Sequential(
5             nn.Conv2d(1, 8, 3, padding=1), nn.ReLU()
6         ),
7         nn.Conv2d(8, 8, 3, padding=1), nn.ReLU()
8         self.pool1 = nn.MaxPool2d(2)
9         self.enc2 = nn.Sequential(
10             nn.Conv2d(8, 16, 3, padding=1), nn.
11             ReLU(),

```

```

12         nn.Conv2d(16, 16, 3, padding=1), nn.
ReLU()
13     )
14     self.pool2 = nn.MaxPool2d(2)
15
16     self.bottleneck = nn.Sequential(
17         nn.Conv2d(16, 32, 3, padding=1), nn.
ReLU(),
18         nn.Conv2d(32, 32, 3, padding=1), nn.
ReLU()
19     )
20
21     self.up2 = nn.ConvTranspose2d(32, 16, 2,
stride=2)
22     self.dec2 = nn.Sequential(
23         nn.Conv2d(32, 16, 3, padding=1), nn.
ReLU(),
24         nn.Conv2d(16, 16, 3, padding=1), nn.
ReLU()
25     )
26
27     self.up1 = nn.ConvTranspose2d(16, 8, 2,
stride=2)
28     self.dec1 = nn.Sequential(
29         nn.Conv2d(16, 8, 3, padding=1), nn.
ReLU(),
30         nn.Conv2d(8, 8, 3, padding=1), nn.ReLU
()
31     )
32
33     self.final = nn.Conv2d(8, 1, 1)
34
35     def forward(self, x):
36         x1 = self.enc1(x)
37         x2 = self.enc2(self.pool1(x1))
38         x3 = self.bottleneck(self.pool2(x2))
39         x = self.up2(x3)
40         x = self.dec2(torch.cat([x, x2], dim=1))
41         x = self.up1(x)
42         x = self.dec1(torch.cat([x, x1], dim=1))
43         return self.final(x).squeeze(1)

```

```

27     binary_image = x
28     x = self.apply_symmetry(x)
29     B, N, C, H, W = x.shape
30     x = x.view(B * N, C, H, W)
31     x = self.unet(x) # run through U-Net
32     x = x.view(B, N, H, W) # unmerge
33     x = self.group_average(x) # average
across symmetry outputs
34     return self.image_mask(x, binary_image.
squeeze(1)) # final masked output

```

### Symmetry, Group Average, Masking

```

1 class ApplySymmetry(nn.Module):
2     def forward(self, x):
3         return torch.stack([
4             x,
5             x.flip(-1),
6             x.flip(-2),
7             x.flip(-1, -2)
8         ], dim=1)
9
10 class GroupAverage(nn.Module):
11     def forward(self, x):
12         return torch.mean(x, dim=1)
13
14 class ImageMask(nn.Module):
15     def forward(self, flow_prediction, input_image
):
16         return torch.abs(flow_prediction *
input_image)
17
18 class UNet_GA(nn.Module):
19     def __init__(self, base_unet):
20         super().__init__()
21         self.apply_symmetry = ApplySymmetry()
22         self.unet = base_unet
23         self.group_average = GroupAverage()
24         self.image_mask = ImageMask()
25
26     def forward(self, x):

```